ST2 Week 4 Tutorial

Task 1:

Player wins:

```
PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 55
Enemy health: 25

ENEMY TURN
Enemy selected:  last stand
Player health: 25
Enemy health: 10

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 10
Enemy health: -20

GAME OVER: Player wins.
```

Draw:

```
ENEMY TURN
Enemy selected:  special
Player health: 20
Enemy health: 15

PLAYER TURN
Select move...
special
You selected:  special
Player health: 25
Enemy health: 5

ENEMY TURN
Enemy selected:  last stand
Player health: -5
Enemy health: -10

DRAW.
```

Enemy wins:

```
ENEMY TURN
Enemy selected:  normal
Player health: 10
Enemy health: 20

PLAYER TURN
Select move...
special
You selected:  special
Player health: 15
Enemy health: 10

ENEMY TURN
Enemy selected:  normal
Player health: -5
Enemy health: 10

GAME OVER: Enemy wins.
```

Task 2.

```
Linear search execution time: 1.295503 seconds
Binary search execution time: 0.000519 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.090928 seconds
Insertion Sort Execution Time: 2.406206 seconds
Binary Search (With Sorting) Execution Time: 2.712254 seconds
Linear Search Execution Time: 0.000097 seconds
```

1. From the screenshot above, we can see that binary search is far quicker at finding an element when compared to linear search. However, with a sorted list we can see that linear search is far quicker than binary search. This is because linear search excels when the list is sorted and binary search excels when the list is not sorted

```
Linear search execution time: 1.765874 seconds
Binary search execution time: 0.000385 seconds
Sorted array: [11, 12, 22, 25, 33, 55, 64, 71, 74, 88]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.790757 seconds
Insertion Sort Execution Time: 2.185796 seconds
Binary Search (With Sorting) Execution Time: 2.092024 seconds
Linear Search Execution Time: 0.000177 seconds
```

We can see that when I make the array bigger, the search time for linear and binary can change. We can see that binary search gets quicker, while linear search time gets longer. This is because there are many more elements in the array for linear search.

Task 3

when array is longer:

```
Linear search execution time: 1.765874 seconds
Binary search execution time: 0.000385 seconds
Sorted array: [11, 12, 22, 25, 33, 55, 64, 71, 74, 88]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.790757 seconds
Insertion Sort Execution Time: 2.185796 seconds
Binary Search (With Sorting) Execution Time: 2.092024 seconds
Linear Search Execution Time: 0.000177 seconds
```

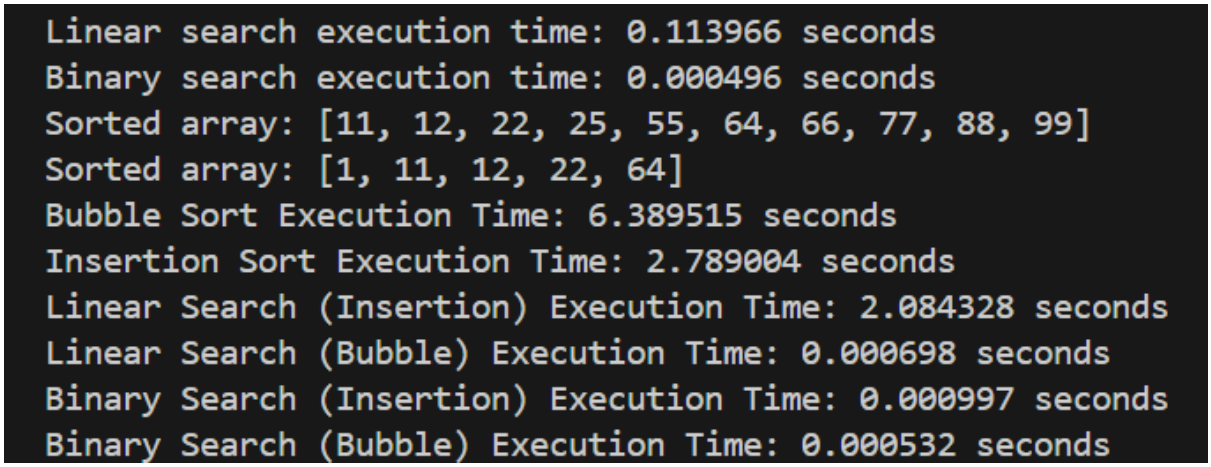
When array is shorter:

```
Linear search execution time: 0.920364 seconds
Binary search execution time: 0.000636 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.293782 seconds
Insertion Sort Execution Time: 2.573560 seconds
Binary Search (With Sorting) Execution Time: 2.031727 seconds
Linear Search Execution Time: 0.000035 seconds
```

4. when the array is longer, we can see that the time it takes for bubble sort to complete is much longer opposed to when the array is shorter. However, insertion sort benefits when the array is longer.

Task 4

1. Below is a screenshot with 4 all functions called



We can see that some search algorithms work better with different sorting algorithms compared to others. Linear search with insertion sort is quite slow, but the other functions are quite efficient.

When comparing these times to the times within take 2, we can see that the times in task 2 are quite different. The only time that was actually efficient in task 2 was the linear search time, the other times took quite long as they were not in the optimal order.